Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Криптография и сетевая безопасность»

Выполнил:
Студент 4-го курса
Группы АС-66
Куган Н.Л.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

Вариант 6

Ход работы

Задание 1

1.Согласно варианту реализовать приложение, генерирующие псевдослучайную равномерно распределенную последовательность произвольной длины из заданного алфавита.

2.Подобрать параметры данного генератора таким образом, чтобы период последовательности имел максимальное значение.

3.Используя статистическое тестирование, проверить гипотезу о том, что сгенерированная последовательность действительно имеет равномерное распределение (см. приложение 2).

4.Отчет должен обязательно содержать несколько тестов, отображающих изменение статистических свойств последовательностей в зависимости от параметров генератора.

Вариант	Генератор	Размер алфавита	Дополнение
6	Линейный конгруэнтный генератор	N=15	*Рассмотреть следующие случаи:* *1. Мультипликативный генератор Лемера: c = 0* *2. Смешанный генератор*

```
class LCG:
    """Линейный конгруэнтный генератор x_{t+1} = (a*x_t + c) mod N.
    c = 0 -> мультипликативный (МКГ, Лемера), c != 0 -> смешанный (СКГ)."""

    def __init__(self, x0, a, c, N):
        if not (0 <= x0 < N and 0 < a < N and 0 <= c < N):
            raise ValueError("Требуется x0, c in A={0..N-1}, a in A\\{0}")
        self.x0, self.a, self.c, self.N = x0, a, c, N
        self.x = x0

    def next(self):
        self.x = (self.a * self.x + self.c) % self.N
        return self.x

    def generate(self, n):
        """Последовательность произвольной длины n: x_1 ... x_n."""
        return [self.next() for _ in range(n)]

    def reset(self):
        self.x = self.x0
```

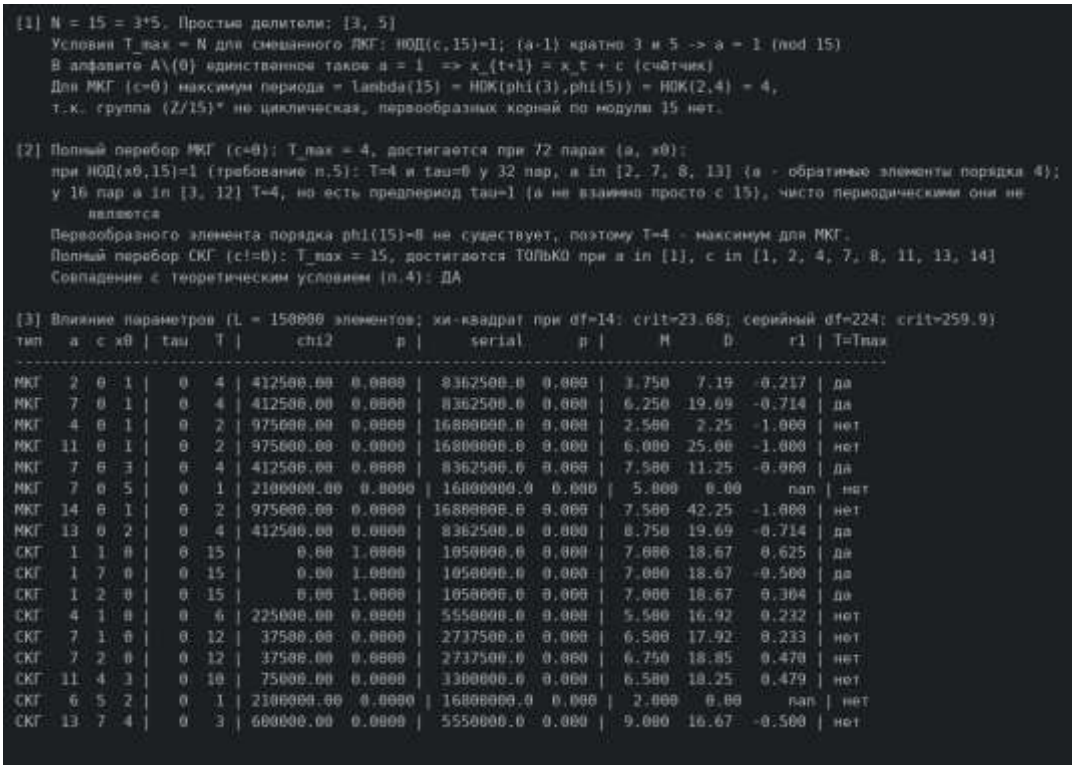
```
def find_period(x0, a, c, N):
    """Возвращает (tau, T): длина предпериода и период последовательности."""
    seen, x, t = {}, x0, 0
    while x not in seen:
        seen[x] = t
        x = (a * x + c) % N
        t += 1
    tau = seen[x]
    return tau, t - tau

def prime_factors(n):
    f, d = set(), 2
    while d * d <= n:
        while n % d == 0:
            f.add(d); n //= d
        d += 1
    if n > 1: f.add(n)
    return f

def hull_dobell(a, c, N):
    """Теоретическое условие T_max = N для ЛКГ (п.4 методички)."""
    a_ok = gcd(c, N) == 1
    b = a - 1
    b_ok = all(b % p == 0 for p in prime_factors(N))
    c_ok = (N % 4 != 0) or (b % 4 == 0)
    return a_ok, b_ok, c_ok, (a_ok and b_ok and c_ok)

def chi_square_uniform(seq, N):
    """Критерий хи-квадрат на равномерность частот элементов алфавита."""
    n = len(seq)
    cnt = Counter(seq)
    obs = np.array([cnt.get(i, 0) for i in range(N)], float)
    exp = n / N
    stat = ((obs - exp) ** 2 / exp).sum()
    p = chi2_dist.sf(stat, N - 1)
    return stat, p, obs

def chi_square_serial(seq, N):
    """Серийный критерий: неперекрывающиеся пары (x_{2i}, x_{2i+1}), N^2 ячеек."""
    pairs = list(zip(seq[0::2], seq[1::2]))
    n = len(pairs)
    cnt = Counter(pairs)
    exp = n / (N * N)
    stat = sum((cnt.get((i, j), 0) - exp) ** 2 / exp for i in range(N) for j in range(N))
    return stat, chi2_dist.sf(stat, N * N - 1), len(cnt)
```



Задание 2

1. Реализовать приложение, позволяющее генерировать простое число по следующей схеме (с использованием, например, теста Рабина - Миллера):

1.Сгенерируйте случайное p-битовое число n.

2.Установите старший и младший биты равными 1. (Старший бит гарантирует требуемую длину простого числа, а младший бит обеспечивает его нечетность.)

3.Убедитесь, что n не делится на небольшие простые числа: 3, 5, 7, 11, и т.д. Во многих реализациях проверяется делимость n на все простые числа, меньшие 256. Наиболее эффективной является проверка на делимость для всех простых чисел, меньших 2000.

Все простые числа, не превосходящие 256 перечислены ниже:
2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,
83,89,97,101,103,107,109, 113,127,131,137,139,149,151,157,
163,167,173,179,181,191,193,197,199,211,223,227,229,233,239, 241,251.

4. Выполните один тест Рабина - Миллера проверки простоты числа для некоторого случайного a. Если n проходит тест, сгенерируйте другое случайное a и повторите проверку. Выбирайте небольшие значения a для ускорения вычислений. Выполните пять тестов. (Одного может показаться достаточным, но выполните пять.) Если n не проходит одной из проверок, сгенерируйте другое n и попробуйте снова.

Можно не генерировать n случайным образом каждый раз, но последовательно перебирать числа, начиная со случайно выбранного до тех пор, пока не будет найдено простое число.

Этап (3) не является обязательным, но это хорошая идея. Проверка, что случайное нечетное n не делится на 3, 5 и 7 отсекает 54 процента нечетных чисел еще до этапа (4). Проверка делимости на все простые числа, меньшие 100, убирает 76 процентов нечетных чисел, проверка делимости на все простые числа, меньшие 256, убирает 80 процентов нечетных чисел. В общем случае, доля нечетных кандидатов, которые не делятся ни на одно простое число, меньшее z, равна 1.12 / ln z.
Вместо теста Рабина - Миллера следует использовать тест, заданный в варианте.
2. В отчете обязательно должны присутствовать тесты, подтверждающие правильность разработанных программ.

```
def primes_upto(n):
    sieve = bytearray([1]) * (n + 1)
    sieve[0:2] = b"\x00\x00"
    for i in range(2, int(n ** .5) + 1):
        if sieve[i]:
            sieve[i * i::i] = bytearray(len(sieve[i * i::i]))
    return [i for i, v in enumerate(sieve) if v]

class BigLCG(LCG):
    """Тот же ЛКГ, но с модулем N = 2^32 и параметрами по теореме Халла-Добелла
    (c нечётно, a-1 кратно 4, T_max = 2^32). Параметры Numerical Recipes.
    Для N = 15 получить p-битовые числа невозможно (15 состояний), поэтому для
    этапа 2 берётся ЛКГ с тем же рекуррентным соотношением и большим модулем.
    Младшие биты ЛКГ с N=2^k имеют короткие периоды, поэтому берутся старшие 16 бит."""

    def __init__(self, seed=None):
        N, a, c = 2 ** 32, 1664525, 1013904223
        assert hull_dobell(a, c, N)[3]
        if seed is None:
            seed = int(time.time_ns()) % N
        super().__init__(seed % N, a, c, N)

    def bits(self, p):
        """p случайных бит из выхода ЛКГ (по 16 старших бит за шаг)."""
        v, got = 0, 0
        while got < p:
            v = (v << 16) | (self.next() >> 16)
            got += 16
        return v >> (got - p)

    def randrange(self, lo, hi):
        """Случайное целое из [lo, hi)."""
        span = hi - lo
        return lo + self.bits(span.bit_length() + 16) % span

def lehmann(n, rng, k=20):
    """Тест Леманна. Возвращает 'composite' или 'probably prime'.
    Для n нечётного: a случайно, t = a^((n-1)/2) mod n.
    t не равно +-1 -> составное. Если t = +-1 во всех k раундах и хотя бы раз t = -1,
    то n простое с вероятностью ошибки <= 2^-k."""
    if n < 2: return "composite"
    if n in (2, 3): return "probably prime"
```

```
    if n % 2 == 0: return "composite"
    seen_minus = False
    for _ in range(k):
        a = rng.randrange(2, n - 1)
        t = pow(a, (n - 1) // 2, n)
        if t == n - 1: seen_minus = True
        elif t != 1: return "composite"
    return "probably prime" if seen_minus else "composite"   # все t=1: не доказано (случай 2^-k для простых)


def generate_prime(p, rng, sieve_bound=2000, k=20, sequential=False):
    """Схема генерации из задания. Возвращает (prime, статистика)."""
    small = primes_upto(sieve_bound)
    st = dict(candidates=0, sieve_rej=0, lehmann_rej=0, lehmann_calls=0)
    t0 = time.perf_counter()
    n = None
    while True:
        if n is None or not sequential:
            n = rng.bits(p)                      # 1. случайное p-битовое число
            n |= (1 << (p - 1)) | 1              # 2. старший и младший биты = 1
        else:
            n += 2                               # перебор подряд нечётных
            if n.bit_length() > p: n = (1 << (p - 1)) | 1
        st["candidates"] += 1
        # 3. просеивание малыми простыми
        if any(n % q == 0 and n != q for q in small):
            st["sieve_rej"] += 1
            continue
        # 4. тест Леманна
        st["lehmann_calls"] += 1
        if lehmann(n, rng, k) == "composite":
            st["lehmann_rej"] += 1
            continue
        st["time"] = time.perf_counter() - t0
        return n, st
```

```
[1] Подробный лог (по одному числу каждого размера, просеивание < 2000)
p=  16 бит | кандидатов    13 | отсеяно делением    12 (92.3%) | отсеяно Леманном   0 | вызовов теста   1 | время
    0.09 мс | sympy.isprime=True
      n = 62539
p=  32 бит | кандидатов    31 | отсеяно делением    30 (96.8%) | отсеяно Леманном   0 | вызовов теста   1 | время
    0.19 мс | sympy.isprime=True
      n = 3716421301
p=  64 бит | кандидатов     2 | отсеяно делением     1 (50.0%) | отсеяно Леманном   0 | вызовов теста   1 | время
    0.31 мс | sympy.isprime=True
      n = 16110992168108487163
p= 128 бит | кандидатов    93 | отсеяно делением    78 (83.9%) | отсеяно Леманном  14 | вызовов теста  15 | время
    1.65 мс | sympy.isprime=True
      n = 245019822550537249385194499337464655491
p= 256 бит | кандидатов    72 | отсеяно делением    62 (86.1%) | отсеяно Леманном   9 | вызовов теста  10 | время
    4.04 мс | sympy.isprime=True
      n = 69736363848422271612...643174115417 (77 десятичных цифр)
p= 512 бит | кандидатов    48 | отсеяно делением    37 (77.1%) | отсеяно Леманном  10 | вызовов теста  11 | время
    21.76 мс | sympy.isprime=True
      n = 13347165476997532321...379435711647 (155 десятичных цифр)
p=1024 бит | кандидатов   274 | отсеяно делением   237 (86.5%) | отсеяно Леманном  36 | вызовов теста  37 | время
    235.81 мс | sympy.isprime=True
      n = 13578320688927804935...565999137511 (309 десятичных цифр)


[3] Проверка корректности теста Леманна (k=20) в сравнении с sympy.isprime
    n = 3..30000 (нечётные): ложных 'простое' = 0, ложных 'составное' = 0
    Числа Кармайкла до 1e5: распознано как составные 16 из 16
    Псевдопростое по основанию 2: 341 -> composite
    Известные простые: 2^61-1 -> probably prime; 2^127-1 -> probably prime
    Составное 2^61+1 -> composite
```